

AI Interview Assistant

Préparation d'entretien assistée par IA

Python 3.12 · FastAPI · SQLAlchemy · scikit-learn · spaCy + Bootstrap · Chart.js
· Docker

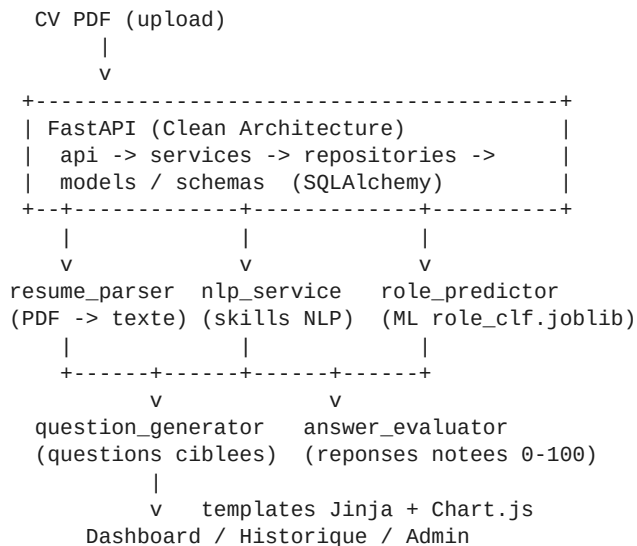


Schéma d'architecture (UI servie par des templates Jinja, serveur requis).

Plateforme web qui aide les candidats à préparer leurs entretiens : parsing du CV PDF, extraction de compétences par NLP, prédiction du rôle par ML, génération de questions ciblées, et évaluation des réponses notées de 0 à 100, le tout sur un dashboard.

1. Clean Architecture (FastAPI)

Séparation stricte en couches `api` / `core` / `services` / `repositories` / `models` / `schemas`. L'app est assemblée dans `app/main.py` via un *lifespan* qui initialise la base et crée l'admin.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    configure_logging()
    init_db()          # create_all (dev) ; Alembic = source de vérité (Docker)
    seed_admin()       # admin@interview.ai / Admin12345!
    yield

app = FastAPI(lifespan=lifespan)
app.mount("/static", StaticFiles(directory=...))
# routes : admin, auth, dashboard, interviews, pages, resumes
```

2. Le pipeline IA (services/)

Service	Rôle
<code>resume_parser.py</code>	Extrait le texte du CV PDF
<code>nlp_service.py</code>	Extraction de compétences (spaCy / NLP)
<code>role_predictor.py</code>	Prédit le rôle (modèle ML <code>role_clf.joblib</code>)
<code>question_generator.py</code>	Génère des questions adaptées au profil
<code>answer_evaluator.py</code>	Note les réponses de 0 à 100

3. Notes de compatibilité (hors Docker)

Lancement prévu via `docker compose up -d`. Testé localement avec `uvicorn` + `SQLite` ; deux correctifs de compat sont dans le code :

- Hachage **bcrypt direct** (sans `passlib`, qui casse avec `bcrypt 5.x`).
- Signature `TemplateResponse(request, name, ctx)` (Starlette 1.3 exige `request` en 1er argument).

```
uvicorn app.main:app --reload
# admin@interview.ai / Admin12345!
```

Dépôt : github.com/sonia633/ai-interview-assistant (public).